A nonuniform Fast Fourier Transform library for GPUs without native double precision

Martin Lachaine
Project page: https://github.com/martinlachaine/mlx-nufft

***Abstract*—Nonuniform fast Fourier transforms (NUFFTs) are used in many imaging, signal-processing and scientific-computing applications. Implementations exist for CPUs and CUDA GPUs, but there is no comparable NUFFT path for Metal GPUs, where native double precision is unavailable. We present a general-purpose NUFFT library for such GPUs, implemented for Apple silicon through MLX. The library supports type 1, type 2, and type 3 transforms in 1D, 2D and 3D. Its main adaptation is a mixed-precision coordinate setup, *crit64*, which performs coordinate rescaling and phase setup in double precision at plan time while maintaining the execution pipeline in single precision. We evaluate the implementation on two Apple silicon Mac systems, an M1 Mac mini with 16 GB of unified memory and an M5 Max MacBook Pro with 128 GB of unified memory. The implementation preserves relative ℓ_2 errors in the 10^{-5} range for the tested cases. On the M5 Max system, type-2 transforms reach $15\times$ and $14\times$ speedups over same-machine CPU FINUFFT in 2D and 3D, respectively. Type-1 spreading is above same-machine CPU FINUFFT on both systems. The slabbed type-3 path runs large anisotropic 3D cases within memory budgets where a direct double-precision CPU grid is not practical.**

***Index Terms*—FFT, NUFFT, GPU, Metal, MLX, mixed precision, single precision**

I. Introduction

The nonuniform fast Fourier transform (NUFFT) generalizes the FFT to data sampled or evaluated away from a Cartesian grid [1], [2], and is often used when Fourier data are naturally non-Cartesian. Common examples include magnetic resonance imaging, optical coherence tomography, cryo-electron microscopy, radio astronomy, diffraction imaging, and particle methods [3], [4]. In the standard terminology, a type-1 NUFFT maps nonuniform source points to uniform Fourier modes; a type-2 NUFFT evaluates a Fourier series at nonuniform target points; and a type-3 NUFFT maps nonuniform source points to nonuniform target frequencies. Direct evaluation of these sums costs $O(MN)$, whereas NUFFT algorithms reduce the cost to nearly linear scaling by combining localized spreading or interpolation with one or more FFTs.

The software ecosystem for NUFFTs is mature on CPUs and on CUDA GPUs. The FINUFFT library provides a general-purpose CPU implementation for transform types 1, 2, and 3 in 1D, 2D and 3D [3]. The cuFINUFFT library extends this line of work to NVIDIA GPUs through CUDA [4]. A separate implementation problem remains for GPUs where CUDA is unavailable and native double precision is absent or impractical. Metal GPUs on current Apple hardware are the motivating example in this work.

Metal GPUs are a useful test case for this implementation problem because they combine substantial GPU throughput and unified memory with no native double precision. A NUFFT implementation for this setting cannot simply be a single-precision translation of existing algorithms. The most sensitive operations occur before the main spreading or interpolation work begins. Coordinate rescaling and phase setup can lose significant accuracy when performed entirely in single precision. The loss becomes apparent for large mode counts and for type-3 transforms whose source and target coordinates span a wide dynamic range. These effects are large enough to make otherwise fast single-precision execution unsuitable for the target tolerances of many imaging and signal-processing workloads.

The implementation described here is a general-purpose NUFFT library for such GPUs. It follows the standard spreading, FFT, and deconvolution structure used by FINUFFT and cuFINUFFT. It uses the same exponential-of-semicircle kernel family as FINUFFT and exposes a FINUFFT-compatible Python interface. The main adaptation is a mixed-precision coordinate setup, called *crit64*. Geometry-dependent coordinate rescaling and phase setup are performed in double precision at plan time. The execution path then stores each grid location as an integer cell index plus a small single-precision local offset. Spreading, FFTs, interpolation, and deconvolution remain in single precision, while the coordinate information most sensitive to roundoff is preserved.

A Metal implementation also requires systems choices that differ from CUDA designs. Metal does not provide the same shared-memory floating-point atomic model used by some CUDA spreading kernels. The implementation therefore separates the type-1 spreading and type-2 interpolation paths since the two operations have different memory access patterns. Type-1 spreading must handle concurrent updates to the fine grid. Type-2 interpolation assigns each nonuniform target point its own local accumulation. The implementation uses point sorting, atomic spreading, and an output-driven spreading path to manage these patterns on Metal. FFTs use MLX's Metal FFT by default, with an optional VkFFT backend [5] for the memory-heavy type-3 stage.

This note describes the algorithmic structure, the Metal implementation, and the measured accuracy, runtime, and memory behavior. The intended accuracy range is approximately 10^{-2} to 10^{-6}, which covers many single-precision imaging and signal-processing workloads. The implementation is not intended to replace native double-precision CPU or CUDA

NUFFTs when users need 10^{-10} to 10^{-12} accuracy. The relevant comparison is the one faced by users on Metal systems today. We compare against same-machine CPU FINUFFT, report accuracy against direct or high-accuracy references, and measure runtime and peak memory on two Apple silicon Mac systems.

II. Algorithms

The implementation follows the standard NUFFT scheme used by FINUFFT and cuFINUFFT. The transform definitions are included here to make the normalization and data movement clear. Let $x_j \in \mathbb{R}^d$ denote nonuniform source or target locations. Let $c_j \in \mathbb{C}$ denote source strengths. Let I denote the requested Cartesian grid of Fourier modes. A type-1 transform maps nonuniform points to uniform Fourier modes,

$$f_k = \sum_{j=1}^{M} c_j e^{\pm ik\cdot x_j}, \qquad k \in I. \tag{1}$$

A type-2 transform applies the adjoint operation and evaluates a Fourier series at nonuniform target points,

$$c_j = \sum_{k\in I} f_k e^{\pm ik\cdot x_j}, \qquad j = 1,\dots,M. \tag{2}$$

A type-3 transform maps nonuniform source points to nonuniform target frequencies s_m,

$$f_m = \sum_{j=1}^{M} c_j e^{\pm is_m\cdot x_j}, \qquad m = 1,\dots,N. \tag{3}$$

For types 1 and 2, the algorithm uses the usual three-stage construction. A type-1 transform spreads each nonuniform source onto an upsampled uniform grid using a localized kernel, applies an FFT on that grid, and deconvolves the transformed grid values to obtain the requested Fourier modes. A type-2 transform applies the adjoint sequence. It pre-scales and embeds the Fourier modes on the upsampled grid, applies an inverse FFT, and interpolates from the grid back to the nonuniform target points.

The spreading kernel is the exponential-of-semicircle kernel used by FINUFFT,

$$\psi(z) = \begin{cases} e^{\beta(\sqrt{1-z^2}-1)}, & |z| \le 1, \\ 0, & |z| > 1. \end{cases} \tag{4}$$

The kernel is applied separably across dimensions. For requested tolerance ε, the default kernel width is

$$w = \lceil \log_{10}(1/\varepsilon) \rceil + 1, \tag{5}$$

with $\beta = 2.30w$ [3], [6]. Types 1 and 2 use an upsampling factor $\sigma = 2$, matching the default high-accuracy setting used by FINUFFT and cuFINUFFT, except that 3D transforms default to $\sigma = 1.25$ at loose tolerances when it is the faster choice, which depends on the tolerance and the point density. The smaller factor shrinks the 3D upsampled grid by about a factor of four and roughly halves execution time on sparse point sets. The tested type-3 slab path uses $\sigma = 1.25$ as described in Section III-C.

A type-3 transform is implemented as the standard composition of type-1 and type-2 ideas [7]. The source coordinates and target frequencies determine an intermediate uniform grid. The sources are rescaled and spread to this grid, an FFT is applied, and the result is interpolated to the nonuniform target frequencies after the required phase and deconvolution factors are applied. The precision-sensitive part of this rescaling is handled by the mixed-precision setup described in Section III-B. The memory-sensitive 3D case is handled by the slabbed path described in Section III-C.

III. Implementation

The implementation targets Metal GPUs through MLX [8]. Custom spreading, interpolation, phase, and setup kernels are written using MLX's Metal kernel interface. FFTs use MLX's Metal FFT by default, with an optional VkFFT backend [5] for the type-3 stage as described below. The execution path uses single-precision complex data. Geometry-dependent setup is separated from execution through a plan object, so repeated transforms with the same point locations can reuse coordinate preprocessing, sorting, kernel parameters, phase factors, and FFT grid choices.

A. Spreading and interpolation

The dominant non-FFT work in a NUFFT is the transfer between nonuniform points and the upsampled grid. Type-1 transforms require spreading from points to the grid. Type-2 transforms require interpolation from the grid to points. Type-3 transforms use both operations. The implementation provides a baseline global-memory atomic spreading path and a bin-sorted path that improves locality by ordering points by their grid neighborhood. Sorting is especially useful for spreading, where nearby points often update nearby grid cells.

Metal imposes a different design point from CUDA for shared-memory spreading. The shared-memory atomic floating-point accumulation strategy used by CUDA NUFFT kernels cannot be translated directly to Metal in the same form. The implementation therefore also includes an output-driven spreading path. The output-driven kernel assigns work by grid tile rather than by source point, accumulates contributions into a padded threadgroup tile, and writes the result back to global memory without requiring floating-point atomics inside the tile. This path is the closest Metal analogue to shared-memory spreading while preserving deterministic ownership of grid outputs within a threadgroup. The output-driven path is the default for type-1 spreading at every kernel width once a plan holds at least 20000 points. Within each tile, points are processed in batches whose separable kernel weights are evaluated cooperatively, once per point and axis, and staged in threadgroup memory, following the subproblem method of cuFINUFFT [4]. The tile accumulates in 32-bit fixed point with native integer atomics, scaled per subproblem so that no cell can overflow, which halved the 3D spreading time. In 1D, a variant gives each threadgroup exclusive ownership of a region of the grid, so that its results are written with plain stores and no atomic operations at all.

Interpolation is simpler to parallelize since each nonuniform target point owns its output value. Each thread reads the neighbouring grid values, evaluates the separable kernel weights, and accumulates a local sum. The same coordinate preprocessing and binning information used by spreading is reused where it improves memory locality.

B. Mixed-precision coordinate setup

The main numerical adaptation is the mixed-precision coordinate setup, which we refer to as *crit64*. A NUFFT execution pipeline can run efficiently in single precision, but coordinate rescaling and phase setup can be overly sensitive to single-precision roundoff. The implementation therefore performs the geometry-dependent rescaling in double precision during planning. Each nonuniform coordinate is converted into an integer grid cell index and a small single-precision offset within that cell. The integer part preserves the large-scale grid location. The single-precision offset preserves the local position used by the spreading or interpolation kernel.

The crit64 representation maintains the execution kernels single precision while avoiding the most damaging loss of information from direct single-precision coordinate rescaling. Phase factors that depend only on the geometry are also prepared during planning and stored as unit complex values in single precision. Runtime execution therefore avoids native double precision and uses only the precomputed local coordinate representation, phase factors, and grid indices.

For type-1 and type-2 plans, the coordinate conversion runs on the GPU by default in double-single arithmetic [9], which reproduces the integer cell indices of the host double-precision conversion exactly. The deconvolution factors require the Fourier transform of the spreading kernel at every mode, and the implementation evaluates it through a Chebyshev approximation that is certified against the reference quadrature to a relative error of 10^{-12} and falls back to the quadrature whenever the certification fails. Together these keep plan construction short, and a 1D plan over 2^{20} modes builds in 22 ms on the M5 Max.

Some applications change point locations between repeated calls while keeping the same mode box and transform type. The implementation supports this case with a GPU-side coordinate re-pointing path. The re-pointing path uses double-single arithmetic in Metal kernels to reproduce the plan-time coordinate setup without returning to a full host-side replan [9]. This path is intended for moving geometry workloads where the cost of rebuilding a full plan would otherwise dominate repeated transforms.

C. Type-3 slab pipeline

The type-3 transform requires a grid whose size is determined by the product of the source and target coordinate extents in each dimension. 3D type-3 problems can therefore produce intermediate grids that are too large for a direct full-grid implementation, especially when one axis is much thinner than the others. The implementation handles this case with a slabbed pipeline.

The slabbed type-3 path chooses a thin axis, processes the intermediate grid in slabs, and accumulates the contribution of each slab to the nonuniform target frequencies. The short transform along the thin axis is fused into the surrounding correction work. The two larger lateral FFTs are then applied slab by slab. This layout keeps peak memory close to the memory required for a small number of grids rather than the full 3D working set. On machines with ample memory, the lateral FFTs run over all slabs at once, which shortens execution at the cost of a higher peak. The approach is especially useful on unified-memory systems, where keeping the computation on the GPU while avoiding excessive scratch allocation is often more important than minimizing one isolated kernel time.

The type-3 path uses an upsampling factor of $\sigma = 1.25$ in this slabbed mode. The smaller upsampling factor reduces memory pressure while maintaining the target single-precision accuracy range described in Section IV.

D. FFT path and memory behavior

The default FFT path uses MLX's Metal FFT. For the non-power-of-two lengths our grids reach, MLX falls back to a large-scratch Bluestein transform, so the library instead uses a four-step Cooley–Tukey split of native FFT calls that avoids it. Type-1 transforms crop each axis to its mode band as soon as that axis has been transformed, and type-2 transforms pad in the mirrored order, so that later axes transform less data and the FFT work in 3D falls to about 58 percent of a full-grid transform. The type-3 slab path additionally offers an optional VkFFT backend [5] through a zero-copy Metal bridge. We use VkFFT specifically there because the two large lateral FFTs dominate the slab cost, where it measured within 4 percent of the MLX path on the FFT stage and on the whole type-3 execute once the four-step split described above was in place. The benchmark results in this note use the MLX default on all transform types. Evaluating VkFFT uniformly as the FFT backend for type-1 and type-2 as well is left to future work, where the FFT is a smaller fraction of the cost and a uniform path would require re-validation across the full benchmark suite. The NUFFT code chooses the upsampled grid sizes, arranges data in the layout the FFT expects, applies phase and deconvolution factors, and manages temporary storage associated with the FFT calls. Explicit transposes in the four-step FFT and the choice of custom Metal kernels over a hand-written FFT are consistent with same-hardware measurements of strided-access cost on these GPUs [10].

FFT memory behavior is still a central implementation concern. Type-1 and type-2 transforms require one upsampled grid. Type-3 transforms can require much larger intermediate grids since the grid size depends on the source and target coordinate extents. The implementation therefore treats FFT grid sizing, scratch allocation, and slab processing as part of the overall NUFFT design rather than as isolated backend details.

Apple silicon Macs use unified memory. The implementation does not have a separate host-to-device transfer stage in the same sense as a discrete CUDA GPU implementation.

Input arrays, output arrays, plans, and intermediate grids still consume memory, but total runtime is not dominated by explicit transfers between CPU and GPU memory. Peak memory remains one of the main constraints for large 3D transforms, which is why the type-3 slab pipeline is part of the default implementation strategy.

E. Plan reuse and interface

The implementation exposes one-shot routines corresponding to the standard `nufft{1,2,3}d{1,2,3}` Python entry points as well as a plan interface for repeated transforms. Plans store the transform type, dimensionality, mode sizes, kernel parameters, coordinate representation, sort order, phase factors, and FFT grid configuration. Reusing a plan avoids repeated coordinate preprocessing and sorting. This is important for workloads that apply the same NUFFT geometry to multiple strength vectors or repeatedly apply adjoint transforms.

The interface is modeled on FINUFFT so that existing Python users can move between CPU and Metal execution with minimal changes. The implementation name and release information are provided in the *Availability* section.

IV. Evaluation

Evaluation of the current implementation is intended to measure accuracy, runtime, and peak memory against FINUFFT on the same machine. This same-machine comparison expected to be a relevant baseline for users on Metal systems, since cuFINUFFT is not available on this hardware. The evaluation is organized by three questions: a) whether the mixed-precision coordinate setup preserves the intended single-precision accuracy range, b) how the implementation performs for type-1 and type-2 transforms across dimensions, and c) whether the type-3 slab pipeline makes large 3D transforms feasible within the available memory.

A. Hardware, software, and protocol

Benchmarks were run on two Apple silicon Mac systems: a) a Mac mini with an M1 SoC and 16 GB of unified memory, and b) a MacBook Pro with an M5 Max SoC and 128 GB of unified memory. Both systems ran macOS 27, MLX 0.31.2, FINUFFT 2.5.1, Python 3.13, and release 0.3.3 of the mlx-nufft library. The CPU baseline on each machine was FINUFFT 2.5.1 using all available CPU cores.

Benchmark sweeps varied transform type, dimension, requested tolerance, point distribution, density, and mode size. The tested distributions were uniform random points on $[-\pi, \pi]^d$ and clustered points in a small box. Density is reported as

$$\rho = \frac{M}{\sigma^d \prod_i N_i}, \tag{6}$$

where M is the number of nonuniform points, N_i is the number of modes in dimension i, and σ is the upsampling factor.

Three timing quantities are reported. The `exec` time measures the transform execution after geometry setup, sorting, and plan construction have already been completed. The `total` time includes plan construction and sorting. The `total+memory` time also includes host and device memory movement where applicable. On Apple silicon Macs, unified memory makes `total+memory` close to `total` for the cases tested, since there is no explicit transfer between separate host and device memories in the usual discrete-GPU sense.

In the type-1 and type-2 sweeps, execution times on the GPU are the median of seven runs that follow a short burst of GPU work and three warm runs, since an idle GPU lowers its clocks and a short transform can otherwise finish before they recover. CPU times in these sweeps are the median of 5 runs after one warm run. The type-3 cases build a plan and execute it once, three times over, and then execute the last plan three more times, on the GPU and on the CPU alike, and report the minimum of each. Each machine's sweep is repeated three times in a single sitting, and every reported value is the median over the repetitions, because single sweeps still differ in their speedups by a median of 8 percent on the M5 Max and 7 percent on the M1. The M1 GPU also moves between performance states over tens of minutes, which changes its absolute execution times by up to 25% between sessions. A fixed calibration workload brackets its sweeps, and in the reported sitting it ran in 24 ms before the first sweep and 25 ms after the last, so the sitting stayed in one performance state. The M5 Max showed only brief excursions of this kind, which the repetition removes.

Plan reuse is important for workloads that apply the same geometry to multiple strength vectors or repeatedly apply adjoint transforms. The `exec` timing therefore reports the cost after coordinate setup, sorting, and FFT planning have already been completed, while `total` includes those setup costs. This separation matches iterative reconstruction and parameter-sweep workloads, where the same nonuniform points are often reused many times.

Accuracy is reported as a relative ℓ_2 error. For type-1 and type-3 transforms the error is

$$\frac{\|\tilde{f} - f\|_2}{\|f\|_2}, \tag{7}$$

where f is the reference output and $\tilde{f}$ is the computed output. For type-2 transforms the same relative error is computed on the nonuniform output vector. The reference is a double-precision direct sum where feasible and a high-accuracy FINUFFT result for larger cases. All GPU execution results use single-precision complex data.

B. Accuracy of the mixed-precision setup

The accuracy study isolates the effect of the mixed-precision coordinate setup. The main failure mode for a direct single-precision implementation is generally not the FFT itself, but the conversion from physical coordinates to fine-grid locations and phase factors. When this conversion is performed entirely in single precision, large mode counts and wide coordinate ranges can lose sufficient information to dominate the final error.

The crit64 setup addresses this failure mode by separating large and small parts of the coordinate representation. The plan stores the integer grid cell exactly and stores only the local offset within that cell in single precision. The execution kernels then use the local offset for kernel evaluation while using the integer cell for indexing. The large coordinate scale is therefore not repeatedly represented as a single single-precision floating-point value during spreading or interpolation.

The effect is clearest in a 1D large-mode test. For a transform with $N = 2^{20}$ modes at requested tolerance $\varepsilon = 10^{-5}$, the Metal implementation reaches a relative ℓ_2 error of 6.9×10^{-6} on both test systems. Single-precision CPU FINUFFT on the same problem reaches only 3.7×10^{-2} on the M1 system and 3.8×10^{-2} on the M5 Max system. The matching Metal results on the two machines show that the remaining error is set by the algorithm and single-precision accumulation rather than by a machine-specific coordinate failure.

A stage-level diagnostic separates the small-coordinate case from the wide-range case. For small coordinates at $\varepsilon = 10^{-5}$ and upsampling $\sigma = 1.25$, an all-double reference pipeline reaches relative ℓ_2 error near 9.3×10^{-6}. An all-single pipeline reaches a floor near 1.4×10^{-5}. Replacing one stage at a time shows that spread accumulation is the most sensitive single-precision stage in this small-coordinate regime, while coordinate rescaling, phase setup, FFTs, interpolation, and deconvolution remain close to the all-double baseline. The wide-range regime changes the dominant error source. Coordinate rescaling becomes the limiting step, which is the regime crit64 is designed to avoid.

The upsampling factor $\sigma = 1.25$ sets a separate limit in single precision. At this factor the Fourier transform of the spreading kernel falls steeply across the band, by a factor of 34 at width 8 and 93 at width 10, and the deconvolution amplifies rounding errors in the single-precision grid by that factor along every axis that carries a full band. Wider kernels therefore stop improving the result, and the implementation caps the width at 8 when all three axes carry a full band, the same limit that single-precision FINUFFT applies, and at 10 otherwise. The achievable error at $\sigma = 1.25$ is then about 6×10^{-5} for requested tolerances of 10^{-5} and below on isotropic 3D problems, in line with single-precision FINUFFT, while thin-slab geometries keep the wider kernel and reach 1.5×10^{-5}. The $\sigma = 2$ paths are unaffected and reach 1.7×10^{-6} at $\varepsilon = 10^{-6}$.

The GPU-side re-pointing path preserves the same accuracy behavior when point locations change between calls. In the tested cases, the double-single Metal setup matches a fresh host-side plan to within the observed run-to-run noise floor. This makes it possible to reuse the mode box and transform configuration while updating coordinates without returning to a full double-precision host setup for every frame.

C. Type-1 and type-2 throughput

The type-1 and type-2 benchmarks measure the two main data movement patterns in the implementation. Type-1 transforms are dominated by spreading from nonuniform points to the upsampled grid. Type-2 transforms are dominated by interpolation from the grid back to nonuniform points. The distinction is important because spreading requires concurrent updates to grid cells, while interpolation assigns each output point to a single thread-local accumulation.

The accuracy and throughput sweep is shown in Figures 1 and 2 for both the M5 Max and M1 systems. The benchmark uses random points at density $\rho = 1$ and compares the Metal implementation with CPU FINUFFT on the same machine. The plotted cost is nanoseconds per nonuniform point as a function of achieved relative ℓ_2 error. Lower values indicate higher throughput.

The signature curves are not monotonic in achieved accuracy, which is expected. The kernel half-width is an integer that grows by 1 for each decade of requested tolerance, so the spreading and interpolation cost rises in discrete steps and only where those stages dominate. This is visible as the clear climb of the 3D type-1 curve, while configurations limited by the FFT or by fixed overhead, such as 1D and 2D type-2, stay nearly flat. The per-call execution times are also short at these point counts, so the measurements carry run-to-run scatter comparable to the small width-dependent increments. The overall trend and the separation between the GPU and CPU curves are robust to this scatter.

The M5 Max system shows the strongest type-2 results. At requested tolerance $\varepsilon = 10^{-5}$, type-2 speedups over same-machine CPU FINUFFT are $4.5\times$ in 1D, $15\times$ in 2D, and $14\times$ in 3D. The 1D case has less work per point and a larger fixed overhead contribution, which reduces the benefit of GPU execution. The 2D and 3D cases expose more interpolation work and make better use of the GPU.

The type-1 results are more sensitive to memory bandwidth and update contention. On the M5 Max system at $\varepsilon = 10^{-5}$, type-1 speedups are $4.8\times$ in 1D, $9\times$ in 2D, and $4.6\times$ in 3D. The same type-1 cases on the M1 system are also above CPU parity, with speedups of $3.3\times$, $3.1\times$, and $2.3\times$ in 1D, 2D, and 3D. The type-2 path is more favorable on both systems in 2D and 3D. On the M1 system, type-2 speedups are $2.3\times$ in 1D, $6\times$ in 2D, and $5\times$ in 3D.

The highest measured on-GPU throughput occurs in the lower-accuracy type-2 regime. The M5 Max system reaches approximately 3.4×10^9 nonuniform points per second in a 2D type-2 case at $\varepsilon = 10^{-1}$. The M1 system reaches approximately 3.7×10^8 nonuniform points per second in a 3D type-2 case at $\varepsilon = 10^{-3}$. These values measure the GPU execution path rather than full plan construction.

Density affects spreading more than interpolation. Figure 3 shows that type-1 speedup on the M5 Max system improves as density decreases. In 2D, the type-1 speedup changes from $7\times$ at $\rho = 10$ to $9\times$ at $\rho = 1$ and $12\times$ at $\rho = 0.1$. In 3D, the corresponding speedups are $3.9\times$, $4.6\times$, and $6.0\times$. Sparse point sets reduce update contention and make the GPU path more favorable. Clustered point distributions are less favorable than uniform random points because they increase locality in the mathematical data while also increasing contention in the

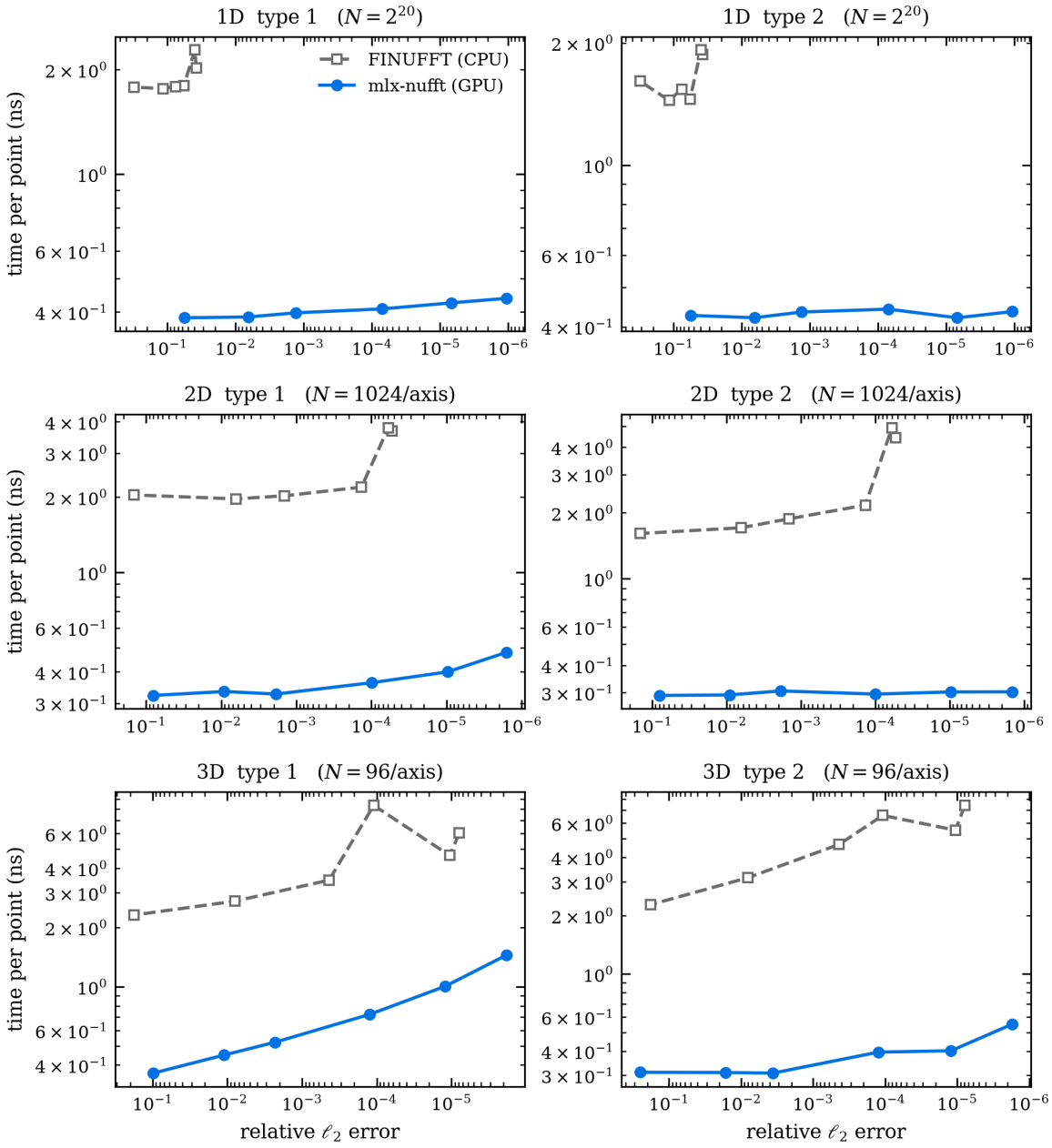


Fig. 1. Time per nonuniform point versus relative ℓ_2 error on the M5 Max for types 1 and 2 in 1D, 2D, and 3D, for the Metal implementation and same-machine FINUFFT, with random points at $\rho = 1$.

grid updates.

D. Type-3 memory and feasibility

The type-3 transform has a different performance profile from types 1 and 2. The intermediate grid is determined by the product of the source and target coordinate extents along each axis. Large coordinate ranges can therefore produce grids that are much larger than the number of source or target points alone would suggest. Memory usage is often the limiting factor before arithmetic throughput becomes the limiting factor.

The evaluation includes two 3D type-3 geometries. The first geometry is a generic case with an intermediate grid of size 640^3. The double-precision CPU grid fits in memory for this case and requires 5.6 GiB. The Metal implementation runs the same family of cases with a peak memory footprint of 4.3 to 5.5 GiB. On the M5 Max system, GPU execution times range from 0.049 s to 0.058 s across the reported target counts. The measured speedups over same-machine double-precision FINUFFT range from $13\times$ to $15\times$ in the representative rows shown in Table I. On the M1 system, the same cases run in 0.58 s to 0.74 s and show speedups from $4.0\times$ to $4.7\times$ in the

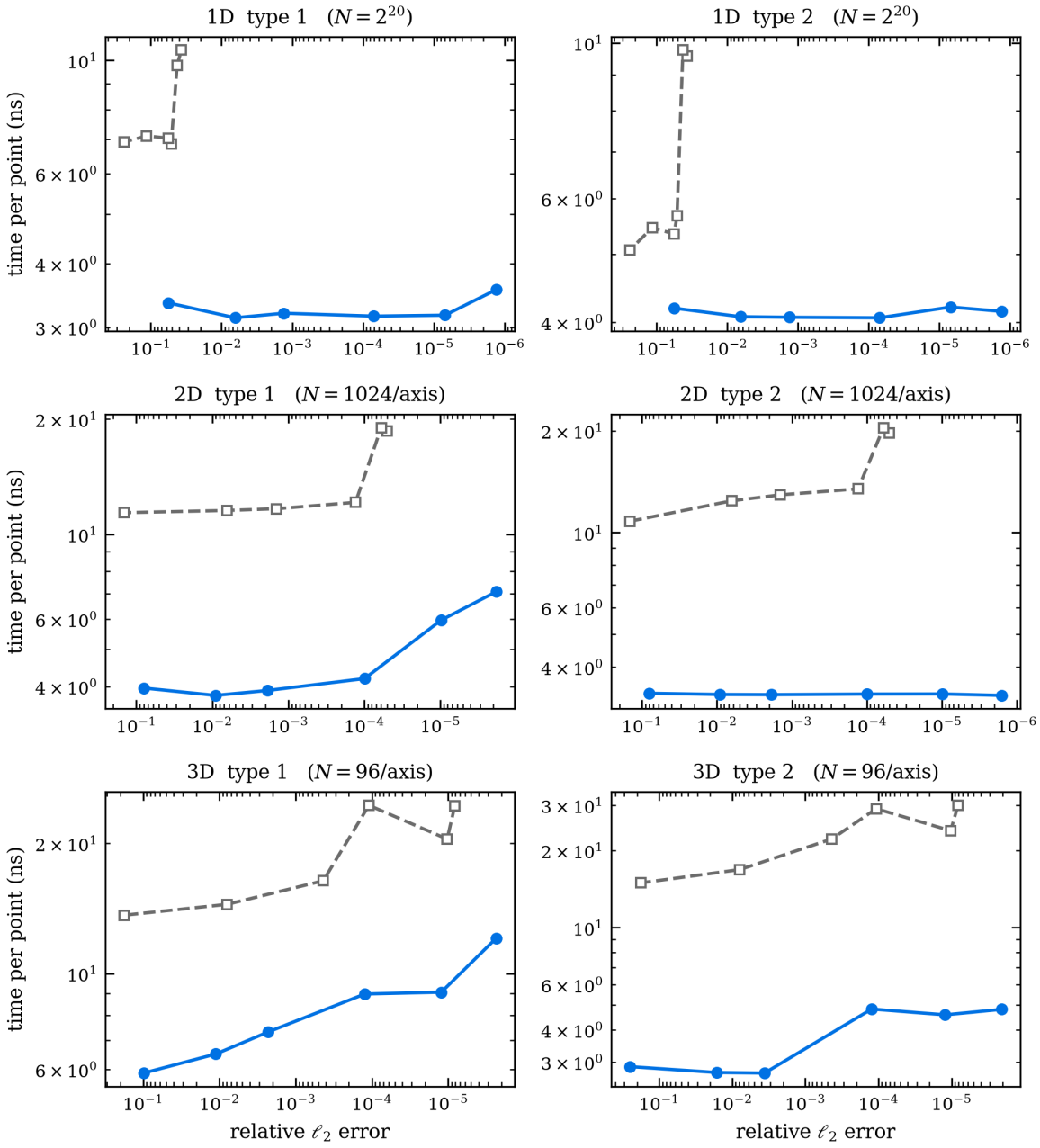


Fig. 2. Time per nonuniform point versus relative ℓ_2 error on the M1, for the same cases as Fig. 1.

representative rows. The corresponding speedups in `total` time, which includes plan construction on both sides and is the cost of transforming a geometry only once, are $6\times$ to $10\times$ on the M5 Max system and $2.3\times$ to $4.3\times$ on the M1 system.

The second geometry is an anisotropic case with an intermediate grid of size $5400^2 \times 24$. The full double-precision CPU grid would require 15.4 GiB. This exceeds the practical memory budget of the 16 GB M1 system and is not a useful CPU baseline there. On the M1 system, the Metal implementation completes the same geometry with a peak memory footprint of 7.8 to 8.0 GiB by using the slabbed type-3 pipeline. The relative ℓ_2 error remains near 1.5×10^{-5} in the tested cases.

On the M1 system, the anisotropic type-3 case is therefore best interpreted as a feasibility result. The slab pipeline does not merely reduce runtime. It changes which problem sizes can be run within the available memory. On the M5 Max system, the largest reported anisotropic case has a GPU execution time near 170 ms. The full wall time is longer because it also includes plan construction, but the execution stage remains small once the geometry has been prepared. The double-precision CPU grid does fit in the memory of the M5 Max

TABLE I

3D TYPE-3 TRANSFORMS WITH THE CRIT64 SINGLE-PRECISION PIPELINE. GPU EXEC TIME, SPEEDUP OVER SAME-MACHINE DOUBLE-PRECISION FINUFFT, AND PEAK UNIFIED MEMORY AGAINST THE MEMORY OF THE DIRECT FP64 CPU GRID. ALL ROWS USE 10^6 NONUNIFORM SOURCES. M IS THE NUMBER OF NONUNIFORM TARGET FREQUENCIES AND EQUALS N^2. CPU OOM MARKS ROWS WHOSE CPU GRID EXCEEDS THE M1'S MEMORY.

			M5 Max (128 GB)		M1 (16 GB)		memory
geometry	fine grid n_{up}	targets $M{=}N^2$	exec (s)	speedup	exec (s)	speedup	peak / CPU fp64
generic	640^3	262k	0.049	15×	0.58	4.7×	5.0 (M1), 5.5 (M5) / 5.6 GiB
	640^3	1.05M	0.050	15×	0.65	4.3×	5.5 (M1), 5.0 (M5) / 5.6 GiB
	640^3	4.19M	0.058	13×	0.74	4.0×	5.3 (M1), 4.3 (M5) / 5.6 GiB
anisotropic	$5400^2 \times 24$	262k	0.17	22×	2.5	CPU OOM	7.8 (M1), 15.7 (M5) / 15.4 GiB
	$5400^2 \times 24$	1.05M	0.17	25×	2.5	CPU OOM	7.8 (M1), 15.7 (M5) / 15.4 GiB
	$5400^2 \times 24$	4.19M	0.17	22×	2.5	CPU OOM	8.0 (M1), 15.9 (M5) / 15.4 GiB

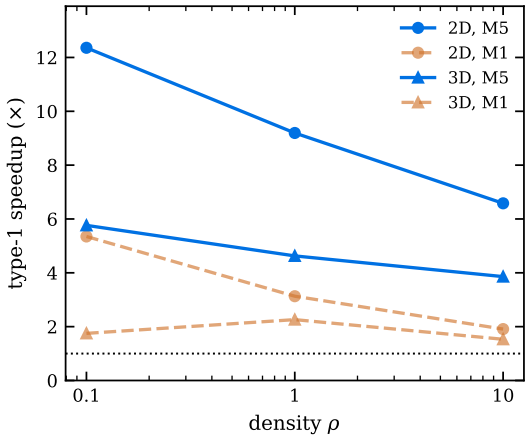


Fig. 3. Type-1 speedup over same-machine CPU FINUFFT as a function of density ρ, at $\varepsilon = 10^{-5}$ with random points, in 2D and 3D on both systems. The dotted line indicates parity with the CPU.

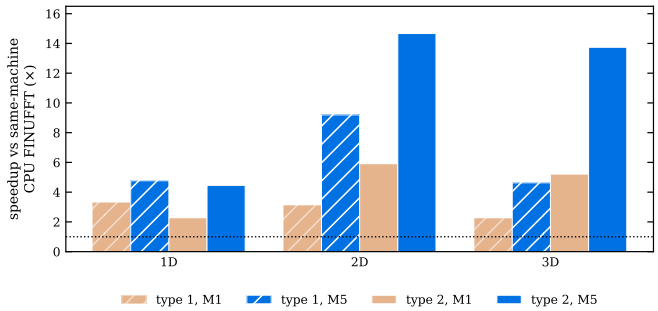


Fig. 4. Speedup over same-machine CPU FINUFFT at $\varepsilon = 10^{-5}$ with random points at $\rho = 1$, by transform type, dimension and system. The dotted line indicates parity with the CPU.

system, where the measured speedups over same-machine double-precision FINUFFT range from 22× to 25× in the anisotropic rows of Table I, and from 18× to 22× in `total` time.

The type-3 results also illustrate why execution time alone is not a sufficient metric for this implementation. A direct full-grid path can have attractive arithmetic structure while still being unusable because of intermediate storage and FFT scratch requirements. The slabbed path trades a more structured execution schedule for lower peak memory. That tradeoff is appropriate for the target systems, where unified memory is large but still finite, and where avoiding excessive temporary allocation is often necessary for large 3D transforms.

E. Cross-machine behavior

The cross-machine comparison shows which parts of the implementation benefit most from the larger Apple silicon GPU. The M5 Max system is faster than the M1 system across the matched GPU execution benchmarks, but the improvement is not uniform across transform types. The strongest difference appears in type-1 spreading, where grid updates and memory bandwidth dominate.

Figure 4 summarizes the type-1 and type-2 speedups across dimensions. Type-1 spreading is above CPU parity on both systems for the tested random cases at $\varepsilon = 10^{-5}$ and $\rho = 1$, by a larger margin on the M5 Max system. The M1 speedups over same-machine CPU FINUFFT are 3.3×, 3.1×, and 2.3× in 1D, 2D, and 3D. The corresponding M5 Max speedups are 4.8×, 9×, and 4.6×. The change reflects the greater memory bandwidth and larger GPU resources of the M5 Max system, which are important for atomic or contention-heavy spreading.

Type-2 interpolation is favorable on both systems. The interpolation kernel assigns one output value to each nonuniform target point and avoids the same pattern of concurrent grid updates. At $\varepsilon = 10^{-5}$ and $\rho = 1$, the M1 system reaches 2.3×, 6×, and 5× speedup in 1D, 2D, and 3D. The M5 Max system reaches 4.5×, 15×, and 14× in the same cases. The 1D M5 Max result is smaller because fixed overhead and FFT cost are a larger part of the total work in that configuration.

The median M5 Max speedup over the M1 system across the matched GPU execution configurations is 10×. The 3D type-3 generic cases show a similar pattern. The M5 Max GPU execution time is 12× to 13× faster than the M1 GPU execution time across the representative rows in Table I.

The accuracy results do not show the same machine dependence. The large-mode crit64 test reaches 6.9×10^{-6} relative ℓ_2 error on both systems. The stage-level precision study also gives matching results on the two machines. These results indicate that the coordinate representation, not machine-specific

arithmetic behavior, controls the achieved accuracy in the tested range. The machine difference is therefore mainly a throughput and memory-capacity difference.

V. Discussion

The evaluation shows that a useful NUFFT implementation is possible on GPUs without native double precision when the precision-sensitive coordinate setup is separated from the main execution path. The resulting implementation is not a double-precision NUFFT. It is a single-precision GPU pipeline with a mixed-precision geometry setup. The design targets workloads where relative errors in the range of approximately 10^{-2} to 10^{-6} are sufficient. Native double-precision CPU and CUDA implementations remain the appropriate tools when users need substantially tighter tolerances.

The main numerical lesson is that single precision is not uniformly inadequate for the NUFFT pipeline. The FFT, interpolation, deconvolution, and much of the spreading work can run in single precision for the target accuracy range. The fragile part is the conversion from user coordinates to fine-grid coordinates and phase factors. The crit64 representation addresses this aspect directly. The large-mode test shows the effect most clearly. A direct single-precision coordinate path loses several digits, while the mixed-precision setup preserves the requested accuracy range on both test machines.

The main performance lesson is that transform type is important. Type-2 interpolation maps well to the tested Metal GPUs because each target point owns its output accumulation. Type-1 spreading is more sensitive to memory bandwidth and update contention because many source points can write to overlapping grid neighborhoods. The M1 and M5 Max results make this distinction visible. Interpolation is favorable on both machines, while spreading is competitive with same-machine CPU FINUFFT on both machines by a smaller margin. Optimizations that reduce arithmetic inside the spreading kernel, such as the piecewise-polynomial kernel evaluation used by cuFINUFFT, did not change spreading time on either GPU, because spreading on these systems is bound by atomic updates and memory traffic rather than by kernel evaluation.

The type-3 results show a different benefit. Large type-3 transforms are often limited by intermediate grid memory rather than by arithmetic throughput alone. The slabbed pipeline reduces peak memory for anisotropic 3D problems and allows cases to run that are not practical with a direct double-precision CPU grid on the smaller tested system. This result is not primarily a peak-speedup result. It shows that a useful problem class can fit within available memory.

Several implementation choices remain open for further tuning. Type-1 spreading is the clearest target because it is the least favorable path on the smaller GPU and the most sensitive to memory bandwidth. More adaptive spreading strategies, improved binning, and additional output-driven kernels may reduce contention. The smaller upsampling factor that 3D type-1 and type-2 transforms already use at loose tolerances could also serve tighter tolerances in memory-limited settings if the accuracy tradeoff is acceptable. The type-3 slab pipeline could be extended with more automatic grid and slab selection. Additional FFT backend tuning may also improve the balance between execution time and peak memory.

VI. Conclusion

This note describes a general-purpose NUFFT implementation for GPUs without native double precision, with Metal GPUs on Apple silicon Macs as the motivating platform. The central design choice is to separate precision-sensitive geometry setup from the single-precision execution pipeline. The crit64 coordinate representation performs coordinate rescaling and phase setup in double precision during planning, then stores grid locations as integer cell indices plus local single-precision offsets. This keeps the spreading, interpolation, FFT, and deconvolution stages in single precision while preserving the coordinate information needed for the target accuracy range.

The evaluation shows that this approach is useful for relative accuracies in the approximately 10^{-2} to 10^{-6} range. Type-2 transforms perform well on both tested Apple silicon Mac systems and reach $15\times$ and $14\times$ speedup over same-machine CPU FINUFFT in 2D and 3D on the M5 Max system. Type-1 spreading is more sensitive to memory bandwidth and is favorable on both systems by a smaller margin. The type-3 slab pipeline reduces peak memory for large anisotropic 3D transforms and makes cases feasible that are not practical with a direct double-precision CPU grid on the smaller tested system.

The implementation is not intended to replace native double-precision CPU or CUDA NUFFTs at tighter tolerances. Its purpose is to provide a documented, inspectable, and usable NUFFT path for Metal GPUs and related hardware where the usual CUDA and double-precision options are not available. Future work includes further tuning of type-1 spreading, more adaptive type-3 slab selection, evaluating the VkFFT backend uniformly across all transform types rather than only the type-3 stage, and continued validation on application workloads.

Availability

The implementation is available under the Apache-2.0 license at https://github.com/martinlachaine/mlx-nufft and on the Python Package Index as `mlx-nufft`, and each release is archived on Zenodo under doi:10.5281/zenodo.21018763. The repository includes installation instructions and examples.

References

[1] A. Dutt and V. Rokhlin, "Fast Fourier transforms for nonequispaced data," *SIAM J. Sci. Comput.*, vol. 14, pp. 1368–1393, 1993.

[2] L. Greengard and J.-Y. Lee, "Accelerating the nonuniform fast Fourier transform," *SIAM Rev. Soc. Ind. Appl. Math.*, vol. 46, pp. 443–454, 2004.

[3] A. H. Barnett, J. Magland, and L. af Klinteberg, "A parallel nonuniform fast Fourier transform library based on an "exponential of semicircle" kernel," *SIAM J. Sci. Comput.*, vol. 41, pp. C479–C504, 2019.

[4] Y.-H. Shih, G. Wright, J. Andén, J. Blaschke, and A. H. Barnett, "cuFINUFFT: a load-balanced GPU library for general-purpose nonuniform FFTs," *IEEE IPDPSW*, 2021.

[5] D. Tolmachev, "VkFFT-A performant, cross-platform and open-source GPU FFT library," *IEEE Access*, vol. 11, pp. 12 039–12 058, 2023.

[6] A. H. Barnett, "Aliasing error of the $\exp(\beta\sqrt{1-z^2})$ kernel in the nonuniform fast Fourier transform," *Appl. Comput. Harmon. Anal.*, vol. 51, pp. 1–16, 2021.
[7] J. Y. Lee and L. Greengard, "The type 3 nonuniform FFT and its applications," *Journal of Computational Physics*, vol. 206, pp. 1–5, 2005.
[8] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, "MLX: Efficient and flexible machine learning on Apple silicon," 2023.
[9] T. J. Dekker, "A floating-point technique for extending the available precision," *Numer. Math. (Heidelb.)*, vol. 18, pp. 224–242, 1971.
[10] M. A. Bergach, "Beating vDSP: A 138 GFLOPS radix-8 Stockham FFT on Apple silicon via two-tier register-threadgroup memory decomposition," *arXiv [cs.DC]*, 2026.